



Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
College of Computer and Information Sciences
Sta. Mesa, Manila

VIRUS INC - ADAPTIVE NETWORK INFECTION

College of Computer and Information Sciences,
Polytechnic University of the Philippines

In partial fulfillment of the course
COSC 304 - Introduction to AI

Bachelor of Science in Computer Science 3-3 | Group #

Biso, Erika P.
Busilacan, Althea May T.
Gente, Daisyrie I.



I. Introduction and Background

Virus Inc. – Adaptive Network Infection is a strategy-simulation game that tasks players with unleashing an evolving computer virus across a complex digital landscape. Instead of tracking a pathogen's movement through physical populations, the game reframes contagion as a battle within a network of devices, emphasising how modern threats propagate through cyberspace. The player starts by infecting a single machine, then strategically navigates through interconnected nodes while evading an intelligent cybersecurity system.

The project takes cues from *Plague Inc.*, where a biological pathogen spreads across the world, and *Bitburner*, an open-source title that turns hacking into an idle programming RPG. But the similarities end there. In *Plague Inc.* the spread is geographic and largely deterministic, while in *Virus Inc.* the infection inhabits a graph of devices and adapts via artificial intelligence rather than scripted behaviour. Likewise, *Bitburner* relies on players writing JavaScript scripts to hack servers, effectively making it a coding game. *Virus Inc.*, by contrast, removes manual coding and empowers the virus to evolve autonomously using AI techniques.

Grounding the simulation in cybersecurity research further elevates the concept. Network attack analysis has shown that uncertain attacker skill and evolving vulnerabilities call for probabilistic modelling, and Bayesian networks are particularly well-suited for analysing attack paths on graphs. Epidemiological tools like the EoN (Epidemics on Networks) library also demonstrate how representing populations as graphs allows flexible stochastic simulations of contagion. These precedents support modelling the game world as a network and inform the probabilistic AI driving both the virus and the defence system.



II. Game Methodology

Objective

The aim of Virus Inc. is to infect at least 90 % of the network before detection reaches 100 %. Players must balance rapid spread with stealth and adaptability. Each infected node yields resources that can be invested to mutate the virus, boosting infection speed, stealth or resistance.

Rules

1. **Network and state:** The network is represented as a weighted graph of devices. Each node has a state (Susceptible, Infected, Patched).
2. **Initial infection:** The game starts with a single infected node chosen by the player. Infection spreads along edges based on the virus's traits and network weights.
3. **Resource economy:** Successful infections generate points that the player spends to evolve traits.
4. **Evolution:** Between turns, the player chooses genetic upgrades to improve speed, stealth or resistance.
5. **Detection:** A cybersecurity AI monitors suspicious traffic. When probabilistic inference suggests a high likelihood of infection on a node, it launches countermeasures, patching devices and raising detection level.
6. **Win/Lose conditions:** If infection reaches 90–100 % of devices before the detection meter fills, the player wins; if detection hits 100 % first, the AI contains the virus and the player loses.

III. Implementation of AI Algorithm

The game integrates three AI algorithms: a Genetic Algorithm for virus evolution, Dijkstra's algorithm for pathfinding and Bayesian inference for the defence system.

Genetic Algorithm (Virus Evolution)

Rationale: Genetic algorithms (GAs) are inspired by natural selection and excel at exploring large search spaces by evolving candidate solutions through selection, crossover and mutation. They have been used



for optimization and machine-learning problems, producing successive generations of solutions that improve fitness. For our game the search space consists of possible virus trait combinations. A GA enables the virus to adapt by evolving traits that maximise spread while minimising detection.

Pseudocode

Initialize a population of candidate virus genomes (sets of trait values).

Evaluate each genome's fitness based on infection success and stealth.

While game continues:

 Select high-fitness genomes to reproduce.

 Perform crossover between pairs to produce offspring.

 Apply random mutations to offspring traits.

 Replace the least-fit genomes with new offspring.

 Use the best genome to update the virus traits in the game world.

Dijkstra's Algorithm (Network Spread)

Rationale: Dijkstra's algorithm is a greedy method that computes the shortest path from a source node to all other nodes and guarantees an optimal path, though it can be slower than heuristic methods. A* search introduces heuristics to speed up execution but may produce non-optimal paths; Dijkstra's is therefore chosen to ensure the virus selects the most efficient routes through the network, maximizing infection spread with minimal detection risk.

Pseudocode

For each node in the network, set distance = ∞ .

Set distance[source] = 0; set all nodes as unvisited.

While there are unvisited nodes:

 Select the unvisited node u with the smallest tentative distance.



For each neighbor v of u :

$\text{NewDist} = \text{distance}[u] + \text{weight}(u, v)$

If $\text{NewDist} < \text{distance}[v]$:

$\text{distance}[v] = \text{NewDist}$

Mark u as visited.

Return the distance table and the paths giving shortest routes to each target.

Bayesian Inference (Cyber Defence)

Rationale: In attack-path analysis for cybersecurity, researchers note that deterministic models cannot capture uncertainty regarding attacker skill and vulnerability exploitation; probabilistic modeling and Bayesian networks better represent uncertain attack environments. The defence AI in our game uses Bayesian inference to estimate the probability that a device is infected based on observable signals (network traffic anomalies) and prior knowledge about infection patterns. When the inferred probability exceeds a threshold, the AI prioritises that node for patching.

Pseudocode

Initialize prior infection probability $P(\text{Infected})$ for each node.

For each time step:

Observe signals S (e.g., unusual traffic, failed logins).

For each node:

Compute likelihood $P(S \mid \text{Infected})$ and $P(S \mid \text{NotInfected})$.

Update posterior $P(\text{Infected} \mid S)$ using Bayes' theorem:

$$P(\text{Infected} \mid S) = \frac{P(S \mid \text{Infected}) * P(\text{Infected})}{[P(S \mid \text{Infected}) * P(\text{Infected}) + P(S \mid \text{NotInfected}) * (1 - P(\text{Infected}))]}$$

If $P(\text{Infected} \mid S) > \text{threshold}$:

Issue patch/containment action on node.

Else continue monitoring.

Decay or adjust priors based on time and actions.



IV. Software Architecture

The software architecture is modular to separate concerns, ensure scalability and facilitate AI integration.

The system comprises the following components:

- **Player Input:** Handles user interactions for upgrades and initial infection choices.
- **Game Controller:** Orchestrates the game loop, dispatching commands to AI systems and updating the game state.
- **AI Systems:** Contains modules for the Genetic Algorithm (evolves virus traits), Dijkstra's pathfinder (calculates optimal infection routes) and Bayesian Defence (estimates infection probabilities).
- **Network Graph System:** Represents the network as a graph of nodes and edges, tracking states and weights.
- **Game State Manager:** Maintains current infections, patch statuses and detection level, updating them based on AI outputs.
- **User Interface:** Visualises the network, infection progress and detection meter for the player.

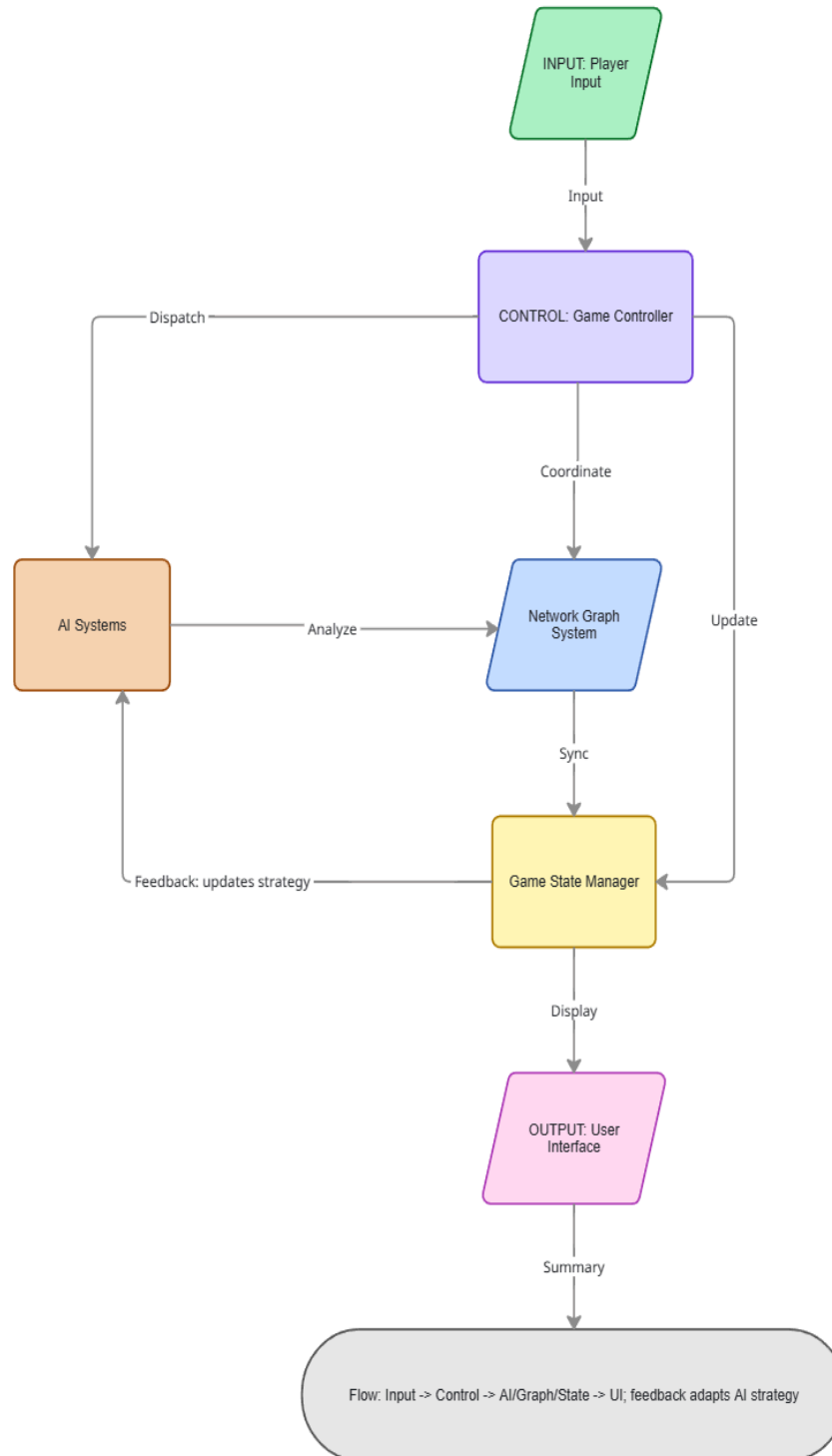


Figure 1. Software Architecture



Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

Sta. Mesa, Manila

Player actions feed into the Game Controller, which invokes the Genetic Algorithm to update virus traits and calls the Dijkstra module to determine the next nodes to infect. The Network Graph System updates the status of nodes as infections occur. Simultaneously, the Bayesian Defence module monitors network events and updates infection probabilities, instructing the Game State Manager to patch nodes when necessary. The Game State Manager records infections, resources and detection level, and the User Interface displays this information to the player.



References:

- Minz, R. L., Pai Nagarmat, S., Rakesh, R., & Isobe, Y. (2018).** *Cyber Security Using Bayesian Attack Path Analysis*. In *CYBER 2018: The Third International Conference on Cyber-Technologies and Cyber-Systems* (pp. 13-68). This paper argues that uncertainty in attacker skills and vulnerabilities encourages probabilistic modeling and that Bayesian networks are well-suited to represent attack graphs—a key idea behind our Bayesian defence system.
- Candra, A., Budiman, M. A., & Hartanto, K. (2020).** *Dijkstra's and A-Star in Finding the Shortest Path: A Tutorial*. IEEE. This tutorial compares Dijkstra's and A* algorithms, noting that Dijkstra's guarantees optimal paths but runs more slowly, whereas A* is faster but not always optimal. These insights justify our choice of Dijkstra's for precise path selection.
- Lambora, A., Gupta, K., & Chopra, K. (2019).** *Genetic Algorithm – A Literature Review*. In *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (Com-IT-Con)* (pp. 380-381). The review explains genetic algorithms as search-optimization methods inspired by natural selection, using selection, crossover and mutation to evolve better solutions over generations—the basis for our virus evolution mechanism.
- Bitburner Official Repository (Fulcrum Games, 2021).** Bitburner is a programming-centric RPG where players hack servers by writing JavaScript scripts and automating tasks. It served as a design contrast: unlike our game, Bitburner relies on manual scripting rather than AI-driven adaptation.
- Epidemics on Networks (EoN) Library Documentation.** The EoN library shows how representing populations as graphs allows flexible stochastic simulation of contagion. This informed our decision to model the game world as a network and to use graph-based simulation techniques.